

1. Diseño del Algoritmo

1.1. Representación de la solución y restricciones

La solución se representa mediante un subconjunto S de localidades, donde cada elemento corresponde al índice de una localidad (de 0 a $n-1$) en la que se ubicará un nuevo centro de urgencias.

- **Restricciones:**
 - El número de nuevos centros debe ser exactamente k .
 - No se puede instalar un centro nuevo en una localidad que ya tenga uno operativo.

El objetivo es minimizar el peor tiempo de acceso (el valor máximo de las distancias mínimas) desde cualquier localidad hasta el centro más cercano.

1.2. Árbol del espacio de soluciones

El árbol de búsqueda sigue una estructura de backtracking:

- **Nivel 0:** Representa el estado inicial del grafo con los centros ya existentes.
- **Nivel i :** Representa la decisión de ubicar el i -ésimo centro nuevo en una de las localidades disponibles.

1.3. Predicados acotadores (Poda)

Para optimizar la búsqueda, se ha implementado un mecanismo de poda:

- En cada nodo del árbol, el algoritmo calcula el peor tiempo de acceso con los centros instalados hasta ese momento.
- Si este tiempo ya es igual o superior al mejor tiempo total encontrado en una rama anterior (`mejor_global`), el algoritmo detiene la exploración de esa rama.
- Como añadir más centros sólo puede reducir el tiempo máximo de respuesta, si la configuración actual ya es peor que nuestra mejor solución completa, es imposible que esa rama produzca un resultado óptimo.

2. Análisis de Resultados

Caso de Prueba	Tiempo Sin Poda (ms)	Nodos Sin Poda	Tiempo Con Poda (ms)	Nodos Con Poda
<i>prueba_pequeña.txt</i>	0.17	17	0.09	3
<i>prueba_trivial.txt</i>	0.04	3	0.04	2
<i>prueba_un_centro.txt</i>	0.05	4	0.06	2
<i>prueba_desconectada.txt</i>	0.06	4	0.05	2
<i>prueba_multiple.txt (1)</i>	0.10	17	0.08	3
<i>prueba_multiple.txt (2)</i>	0.01	3	0.02	2
<i>prueba_enunciado.txt (1)</i>	0.05	5	0.06	2
<i>prueba_enunciado.txt (2)</i>	0.10	17	0.07	3
<i>prueba_grande.txt</i>	59.58	1465	1.57	4

1. **Eficacia de la poda:** La reducción de nodos en el caso *prueba_grande.txt* (de 1465 a solo 4) demuestra que la poda es crítica para manejar volúmenes de datos grandes.
2. **Rendimiento:** El tiempo de ejecución en la versión con poda se mantiene estable incluso al aumentar la complejidad, mientras que en la versión sin poda aumenta exponencialmente.
3. **Anomalías en casos pequeños:** En pruebas muy simples como *prueba_un_centro.txt*, la versión con poda puede tardar ligeramente más debido al coste de calcular las cotas y ordenar las opciones.

